

Fiche TP — Semaine 2

Structure HTML du MiniForum

Développement web, classe passerelle

Durée 4h

Rendu avant S3

Objectif du TP

Construire les **deux pages** du MiniForum en HTML pur : la page d'accueil, avec sa liste de sujets, et la page de détail d'un sujet, avec ses messages. Tout est codé en dur, et il n'y a **aucun style** — c'est voulu.

Avant de commencer

Ce TP suppose que les notions du polycopié *S02_polycop* (structure HTML5, imbrication, éléments sémantiques, liens, chemins relatifs) ont été vues en cours. Garde-le à portée de main.

Tu travailles dans le dépôt miniforum créé au TP de la semaine 1.

Comment lire cette fiche

Cette fiche ne donne pas le code à recopier. Elle décrit le **rendu attendu** sous forme de maquette, et précise ce qu'il faut produire. À toi d'écrire le HTML qui y correspond, en t'appuyant sur le polycopié.

Les balises annotées sur les maquettes (`<header>`, `<article>`...) sont des indications, pas du texte à afficher.

Étape		Ce que tu produis	Temps
1–2	Reprise et squelette	la charpente de <code>index.html</code>	30 min
3–5	En-tête, sujets, pied de page	la page d'accueil complète	1h30
6–7	Page de détail et liens	<code>sujet.html</code> et la navigation	1h15
8	Validation et envoi	un dépôt propre et validé	45 min

1 Reprendre le projet

1. Ouvrir le dossier miniforum dans VS Code (*Fichier → Ouvrir le dossier*).
2. Ouvrir un terminal intégré (*Terminal → Nouveau terminal*) et récupérer les éventuelles mises à jour :

```
git pull
```

3. Ouvrir `index.html`, créé la semaine dernière. Son contenu va être **entièrement remplacé** par la vraie structure du forum.

Le réflexe Live Server

Clic droit sur `index.html` → *Open with Live Server* (extension installée en semaine 1). La page s'ouvre dans le navigateur et se **recharge automatiquement** à chaque sauvegarde. Garde-la ouverte pendant tout le TP, à côté de VS Code.

2 Poser le squelette sémantique

On commence par la charpente, sans contenu : découper la page en zones avant de les remplir.

1. Remplacer le contenu de `index.html` par ce squelette :

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>MiniForum - Accueil</title>
7 </head>
8 <body>
9   <header>
10    <!-- Titre du site + navigation -->
11  </header>
12
13  <main>
14    <!-- Liste des sujets -->
15  </main>
16
17  <footer>
18    <!-- Pied de page -->
19  </footer>
20 </body>
21 </html>
```

Extrait 1 – `index.html` — la charpente

2. Sauvegarder (Ctrl+S) et regarder le navigateur : la page est **vide**. Seul l'onglet affiche « MiniForum - Accueil ».

Pourquoi une page vide ?

`<header>`, `<main>` et `<footer>` ne produisent rien de visible par elles-mêmes : elles **structurent** le document. C'est le contenu qu'on va y placer qui s'affichera.

Un premier commit tout de suite

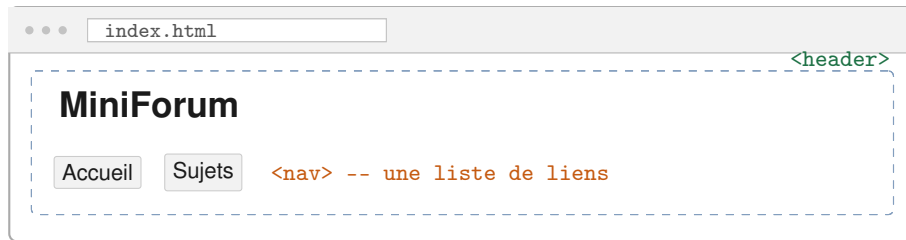
Ne garde pas ton travail dans un seul commit final. Dès que la charpente est en place :

```
git add index.html
git commit -m "Charpente sémantique de la page d'accueil"
```

L'historique Git fait partie de l'évaluation du projet final.

3 Remplir l'en-tête

Dans le `<header>`, on place le titre du site et la barre de navigation. Voici le **rendu attendu** :



À produire

Un `<header>` contenant :

- un titre `<h1>` « MiniForum » — **le seul** `<h1>` de la page ;
- un `<nav>` contenant un `<ul>` de deux `<li>`, chacun avec un lien :
 - « Accueil » vers `index.html`
 - « Sujets » vers `#sujets` — la section que tu créeras à l'étape suivante

Un menu est une liste — même si ça surprend

Il serait plus court d'écrire les deux liens à la suite, sans `<ul>`. Mais un menu **est** une liste de liens : c'est ce que doivent comprendre les lecteurs d'écran, et c'est ce que le CSS mettra en forme horizontalement en semaine 4.

Attention à l'imbrication : le `<a>` va **dans** le `<li>`, jamais l'inverse.

Le lien avec le

`href="#sujets"` est un **lien interne** : il pointe vers l'élément de la page dont l'id vaut `sujets`. Tant que cet élément n'existe pas, cliquer dessus ne fait rien — c'est normal. Reviens le tester une fois l'étape 4 terminée.

4 La liste des sujets

C'est le cœur de la page d'accueil. Chaque sujet du forum est un `<article>` : rappelle-toi, un article garde du sens sorti de son contexte.

Dans le `<main>`, ajoute une `<section>` contenant trois sujets codés en dur. Voici le **rendu attendu** :

À produire

Une `<section id="sujets">` contenant :

- un titre `<h2>` « Sujets récents » ;
- **trois** `<article>`, chacun composé d'un titre `<h3>`, d'une ligne d'information (auteur en `<strong>` et catégorie) et d'un paragraphe `<p>`.

Le titre `<h3>` du **premier** sujet doit être un lien vers `sujet.html` — la page que tu créeras à l'étape 6.

Vérifie ta hiérarchie de titres

La page contient maintenant un `<h1>` (le nom du site), un `<h2>` (« Sujets récents ») et trois `<h3>` (les sujets). C'est exactement la structure attendue : **un seul** `<h1>`, et **aucun saut de niveau**.

Si tu as choisi un `<h3>` plutôt qu'un `<h2>` « parce que c'était plus petit », reprends : la taille se réglera en CSS.

« Codé en dur » — pourquoi ?

Ces trois sujets sont écrits directement dans le HTML : ils ne viennent d'aucune base de données. Quoi que tu fasses, ce sont toujours les mêmes. C'est la limite volontaire de la phase 1 — elle rendra évident, en semaine 8, le besoin d'une vraie base de données.

5 Le pied de page

MiniForum — Projet du cours de développement web
Classe passerelle — cfpt-i

<footer>

À produire

Un <footer> avec deux paragraphes <p> : le nom du projet, et la mention de la classe.

Point d'étape

La page d'accueil est terminée. Commite avant de passer à la suite :

```
git add index.html
git commit -m "Page d'accueil : en-tete, liste de sujets, pied de
page"
```

6 La page de détail d'un sujet

Un forum ne se limite pas à une liste : cliquer sur un sujet doit ouvrir la discussion. C'est la seconde page du MiniForum.

1. Créer un nouveau fichier `sujet.html`, à la racine du dossier `miniforum` (à côté de `index.html`).
2. Y reprendre le même squelette et le même `<header>` que dans `index.html` — copier-coller est ici légitime : les deux pages partagent la même charpente.
3. Changer le `<title>` en « MiniForum - Bienvenue sur le MiniForum ».

À produire dans le `<main>` de `sujet.html`

- un lien de retour vers `index.html`, avec un intitulé explicite ;
- un `<h2>` reprenant le titre du sujet, suivi d'un `<p>` d'information (auteur, date, catégorie) ;
- une `<section>` intitulée par un `<h3>` « Réponses », contenant **trois** `<article>` ;
- dans chaque `<article>` : un `<p>` avec l'auteur en `<strong>` et la date, puis un `<p>` avec le texte du message.

Un message est bien un `<article>`

La question revient chaque année : pourquoi `<article>` pour un simple message ? Applique le test du photocopié — ce message garderait-il du sens, cité ailleurs ? Oui. C'est donc un contenu autonome, et le `<article>` est le bon choix.

La `<section>` « Réponses », elle, n'a de sens que dans cette page : elle regroupe par thème.

Pour aller un cran plus loin sur les dates

La balise `<time>` permet d'indiquer une date sous une forme lisible par une machine, tout en affichant le texte de ton choix :

```
<time datetime="2026-09-16">16 septembre 2026</time>
```

Ce n'est pas obligatoire ici, mais c'est exactement le genre de détail qui distingue un HTML sémantique d'un HTML approximatif.

7 Relier les deux pages

Les deux fichiers existent ; il reste à pouvoir circuler de l'un à l'autre.

1. Dans `index.html`, vérifier que le titre du premier sujet est bien un lien vers `sujet.html`.
2. Dans `sujet.html`, vérifier que le lien de retour pointe vers `index.html`.
3. Tester l'aller-retour dans le navigateur : accueil → sujet → accueil.
4. Tester aussi le lien `#sujets` du menu : la page doit défiler jusqu'à la liste des sujets.

Si un lien ne fonctionne pas

Dans neuf cas sur dix, le chemin est en cause. Vérifie dans l'ordre :

- le nom du fichier est exact, **casse comprise** (sujet.html, pas Sujet.html) ;
- les deux fichiers sont bien dans le **même dossier** ;
- le chemin est **relatif** (sujet.html) et non absolu (C:/Users/...) ;
- pour le lien interne, l'id visé existe et s'écrit **sans** le # dans l'attribut id.

8 Valider et sauvegarder

Valider **les** **deux** **pages**

1. Aller sur validator.w3.org, onglet *Validate by Direct Input*.
2. Copier-coller tout le contenu de `index.html`, puis cliquer sur *Check*.
3. Corriger les erreurs signalées, en commençant par **la première de la liste**, et relancer la validation.
4. Recommencer l'opération avec `sujet.html`.

Rappel de méthode

Une seule balise oubliée peut produire dix messages en cascade : corrige la première erreur, revalide, et le compteur chute souvent d'un coup. Les **Errors** doivent toutes disparaître ; les **Warnings** signalent une maladresse sans invalider la page.

Message	Correction
Unclosed element X	Fermer la balise X
Stray end tag X	Supprimer la fermante en trop, ou ajouter l'ouvrante
Duplicate ID "sujets"	Deux éléments portent le même id : en renommer un
Element p not allowed as child of element ul	Le <p> doit être <i>dans</i> le
Consider adding a lang attribute	Ajouter <code>lang="fr"</code> sur la balise <html>

Envoyer **sur** **GitHub**

```
git add .
git commit -m "Page de detail d'un sujet et navigation entre les pages"
git push
```

Vérifier que c'est bien parti

Rafraîchis la page de ton dépôt sur github.com : `index.html` et `sujet.html` doivent y apparaître, et l'onglet *Commits* doit montrer **plusieurs** commits pour cette séance — pas un seul.

9 Pour aller plus loin (optionnel)

Si tu as terminé en avance :

- Ajouter une image de logo dans le <header> des deux pages, avec un alt correct, rangée dans un sous-dossier `images/`.

- Transformer les titres des deux autres sujets en liens vers `sujet.html` également.
- Ajouter dans `index.html` un tableau récapitulatif des sujets (titre, auteur, nombre de réponses) avec `<caption>`, `<thead>` et `<tbody>`.
- Utiliser la balise `<time>` avec son attribut `datetime` pour toutes les dates.
- Ajouter un `<aside>` dans `sujet.html` : « Sujets similaires », avec une liste de deux liens.

Toujours pas de CSS

La tentation est grande d'ajouter quelques couleurs. Résiste : une page sans style force à juger la **structure**, et c'est précisément ce qui est évalué cette semaine. Le CSS commence en semaine 4, et il s'appliquera sans que tu aies à retoucher ce HTML.

10 Points de contrôle

Auto-évaluation

- ☐ `index.html` et `sujet.html` existent, à la racine de `miniforum`
- ☐ Les deux pages ont un `<!DOCTYPE html>`, un `lang="fr"` et un `<meta charset="UTF-8">`
- ☐ Chaque page a une structure sémantique : `<header>`, `<main>`, `<footer>`
- ☐ L'en-tête contient un `<h1>` et un `<nav>` avec une liste de liens
- ☐ La page d'accueil affiche **trois** sujets, chacun dans un `<article>`
- ☐ La page de détail affiche **trois** messages, chacun dans un `<article>`
- ☐ Il y a **un seul** `<h1>` par page, et aucun saut de niveau de titre
- ☐ Le code est **indenté** : un niveau par niveau d'imbrication
- ☐ L'aller-retour accueil ↔ sujet fonctionne, ainsi que le lien `#sujets`
- ☐ Toute image éventuelle a un attribut `alt` qui décrit quelque chose
- ☐ Les deux pages passent `validator.w3.org` **sans erreur**
- ☐ Le travail est poussé sur GitHub, en **plusieurs** commits au message clair

À rendre avant la semaine 3

Le dépôt GitHub `miniforum` à jour, avec les deux pages validées. La semaine 3 ajoutera les **formulaires** et les bases de l'**accessibilité** — ce sera la dernière séance de HTML avant le CSS.

Bloqué ?

Si une étape résiste plus de dix minutes, demande. Le polycopié *S02_polycop* contient la réponse à la quasi-totalité des questions de ce TP — commence par y chercher, puis appelle.